Call Me Maybe

Cours 5 — JSON et Pydantic

---

1. Les fichiers du projet

Ton programme lit deux fichiers JSON en entree et produit un fichier en sortie.

```
data/
  input/
    function_definitions.json    # les fonctions disponibles
    function_calling_tests.json  # les prompts a traiter
  output/
    function_calling_results.json  # ta sortie
```

2. Lire et ecrire du JSON en Python

```
import json
# Lire un fichier JSON
with open('data/input/function_definitions.json') as f:
    data = json.load(f)           # -> dict ou list Python
# Ecrire un fichier JSON
with open('data/output/results.json', 'w') as f:
    json.dump(results, f, indent=2)
# Gerer les erreurs (obligatoire dans le projet !)
try:
    with open(path) as f:
        data = json.load(f)
except FileNotFoundError:
    print('Erreur : fichier introuvable')
except json.JSONDecodeError as e:
    print(f'Erreur : JSON invalide — {e}')
```

Le sujet precise que les fichiers peuvent etre manquants ou invalides. Toutes les erreurs doivent etre gerees — un crash non gere = projet non fonctionnel.

3. Qu'est-ce que Pydantic ?

Sans Pydantic, `json.load()` retourne un dictionnaire Python brut. Aucune garantie que les champs existent, que les types sont corrects, ou que le schema est conforme. Pydantic permet de definir un **schema** sous forme de classe Python et de valider automatiquement les

donnees contre ce schema.

> ***Un schema = la structure attendue de tes donnees.***
>
> `Exemple : {"prompt": "..."} -> schema valide pour TestPrompt Exemple : {"question": "..."} -> schema invalide (mauvaise cle) Exemple : {"prompt": 42} -> schema invalide (mauvais type)`

4. Les modeles Pydantic pour le projet

Classe	Fichier associe	Champs principaux
`ParameterDef`	function_definitions.json	`type: str # 'number', 'string', 'boolean'`
`FunctionDef`	function_definitions.json	`name: str, description: str, parameters: dict[str, ParameterDef], returns: ParameterDef`
`TestPrompt`	function_calling_tests.json	`prompt: str`
`FunctionCall`	function_calling_results.json (sortie)	`prompt: str, name: str (min_length=1), parameters: dict`

```
from pydantic import BaseModel, Field
from typing import Optional
class ParameterDef(BaseModel):
    type: str                              # 'number', 'string', 'boolean'
class FunctionDef(BaseModel):
    name: str
    description: str
    parameters: dict[str, ParameterDef]   # {'a': {'type': 'number'}, ...}
    returns: ParameterDef
class TestPrompt(BaseModel):
    prompt: str
class FunctionCall(BaseModel):
    prompt: str
    name: str = Field(..., min_length=1)   # ne peut pas etre vide
    parameters: dict
```

5. Utilisation pratique

```
from pydantic import ValidationError
def load_function_definitions(path: str) -> list[FunctionDef]:
    """Charge et valide les definitions de fonctions."""
    try:
        with open(path) as f:
            raw = json.load(f)            # -> list de dicts
    except FileNotFoundError:
        raise FileNotFoundError(f'Fichier introuvable : {path}')
    except json.JSONDecodeError as e:
        raise ValueError(f'JSON invalide dans {path} : {e}')
    functions = []
    for item in raw:                      # raw est une liste de dicts
        try:
            functions.append(FunctionDef(**item))  # validation Pydantic
        except ValidationError as e:
            raise ValueError(f'Definition invalide : {e}')
    return functions
# Utilisation
functions = load_function_definitions('data/input/function_definitions.json')
for func in functions:
    print(func.name)                      # garanti str
    for param_name, param_def in func.parameters.items():
        print(f'  {param_name}: {param_def.type}')  # garanti str
```

6. Serialiser vers JSON (le fichier de sortie)

```
# Creer un resultat
result = FunctionCall(
    prompt='What is the sum of 2 and 3?',
    name='fn_add_numbers',
    parameters={'a': 2.0, 'b': 3.0}
)
# Convertir en dict -> JSON
result.model_dump()
# -> {'prompt': '...', 'name': 'fn_add_numbers', 'parameters': {'a': 2.0, 'b': 3.0
}}
# Ecrire tous les resultats
results = [result1, result2, result3]
with open('data/output/function_calling_results.json', 'w') as f:
    json.dump([r.model_dump() for r in results], f, indent=2)
```

7. Comportement important : Pydantic v2 et les types

Par defaut, Pydantic v2 **convertit** les types compatibles au lieu de planter. C'est un piege courant :

```
class FunctionDef(BaseModel):
    name: str
# Pydantic v2 convertit silencieusement 42 en '42'
f = FunctionDef(name=42)     # PAS d'erreur !
print(f.name)                # -> '42'  (string !)
# Pour interdire ca, utiliser le mode strict :
from pydantic import ConfigDict
class FunctionDef(BaseModel):
    model_config = ConfigDict(strict=True)
    name: str
f = FunctionDef(name=42)     # -> ValidationError !
```

Dans le projet, le mode strict n'est pas obligatoire mais il rend ton code plus robuste.

8. Les erreurs a gerer — recap complet

#	Exception	Cause	Comment gerer
1	`FileNotFoundError`	Fichier JSON manquant	Afficher message clair, arreter proprement
2	`json.JSONDecodeError`	Fichier existe mais JSON syntaxiquement invalide	Afficher ligne et colonne de l'erreur
3	`ValidationError (Pydantic)`	JSON valide mais schema incorrect (champ manquant ou mauvais type)	Afficher quel champ pose probleme
4	`ValueError (get_valid_tokens)`	Liste de tokens valides vide (etat JSON incoherent)	Logger l'erreur, passer au prompt suivant

---

Corrections de l'exercice

Erreurs dans le code soumis :

Erreur	Code incorrect	Code correct
Typo	`parameter: dict`	`parameters: dict (nom exact impose par le sujet)`
list(Test Prompt)	`res = list(TestPrompt)`	`prompts = [] (liste vide, pas list(classe))`
Iteration	`for prompt in f: res.append(prompt.prompt)`	`raw = json.load(f) for item in raw: prompts.append(TestPrompt(**item))`

Erreur	Code incorrect	Code correct
ValidationError manquant	`# pas de bloc except ValidationError`	`except ValidationError as e: raise ValueError(f'Schema: {e}')`

Code corrige complet :

```
def load_test_prompts(path: str) -> list[TestPrompt]:
    """Charge et valide les prompts de test."""
    try:
        with open(path, 'r') as f:
            raw = json.load(f)             # parse le JSON entier
    except FileNotFoundError:
        raise FileNotFoundError(f'Fichier introuvable : {path}')
    except json.JSONDecodeError as e:
        raise ValueError(f'JSON invalide dans {path} : {e}')
    prompts = []                           # liste vide
    for item in raw:                       # raw = list de dicts
        try:
            prompts.append(TestPrompt(**item))
        except ValidationError as e:
            raise ValueError(f'Schema invalide : {e}')
    return prompts
```

A retenir

- json.load() retourne un dict brut sans validation.
- Pydantic valide automatiquement le schema via FunctionDef(**item).
- Pydantic v2 convertit les types compatibles par defaut (42 -> '42').
- Toujours intercepter : FileNotFoundError, JSONDecodeError, ValidationError.
- model_dump() convertit un objet Pydantic en dict pour json.dump().

Prochaine etape

Cours 6 — Implementer le constrained decoding : ecrire la boucle de generation complete, manipuler les logits avec numpy, et construire get_valid_tokens() pour chaque etat de la machine.